LLM Capability Map: Where Models Excel vs Struggle

The Core Principle

LLM capability at any task is a function of three factors:

1. **Training data density** — How much high-quality example code for this task/language/pattern exists in the training corpus
2. **Pattern regularity** — How predictable and template-like the task is (high regularity = strong statistical prior)
3. **Verifiability during RL** — Whether the task could be verified with automated tests during RLVR training (verified = model learned to reason about it)

If all three are high, the model is exceptional. If any is low, performance degrades. This framework predicts almost every strength and weakness below.

By Programming Language

Tier 1: Exceptional (massive training data, heavily used in RL)

Language	Why It's Strong	Practical Notes
Python	Dominant in training data, GitHub, Stack Overflow, tutorials, docs. SWE-bench is entirely Python. RLVR training primarily uses Python test execution.	Best overall language for LLM-assisted coding. Claude is near-expert level.
JavaScript / TypeScript	Second most represented language on GitHub. Enormous ecosystem of tutorials, docs, npm packages. TypeScript's type annotations provide extra statistical signal.	Excellent. TypeScript often produces better results than plain JS because types constrain the output space.
Java	Massive enterprise codebase representation. Verbose, highly structured syntax means strong patterns. Extensive documentation culture.	Very strong for standard enterprise patterns (Spring Boot, Maven, JUnit). The verbosity that annoys humans actually helps the model — more tokens = more signal.
C / C++	Large training representation. Well-documented, decades of tutorials	Strong for standard patterns. Struggles more with complex

Language	Why It's Strong	Practical Notes
	and textbooks.	memory management, undefined behavior, and platform-specific nuances.
Go	Well-documented, opinionated language with consistent style. Strong GitHub presence.	Very good. Go's simplicity and enforced conventions create highly predictable patterns the model excels at.
Rust	Growing rapidly in training data. Strong documentation culture. Compiler-enforced patterns.	Good and improving fast. The borrow checker creates very specific patterns the model can learn. However, complex lifetime annotations can still trip it up.
SQL	Enormous representation. Highly structured, rule-based syntax.	Excellent for standard queries. Degrades on complex multi-CTE analytical queries or database-specific dialects.

Tier 2: Good (solid training data, some gaps)

Language	Why It's Weaker	Practical Notes
C#	Good representation but less than Python/JS/Java. .NET ecosystem well-documented.	Strong for standard ASP.NET, Entity Framework, LINQ patterns. May confuse .NET Framework vs .NET Core conventions.
Ruby	Rails dominated training data. Less representation of modern Ruby outside Rails.	Very strong for Rails patterns specifically. Weaker for non-Rails Ruby.
PHP	Large but lower-quality training data (many old tutorials, legacy code). Modern PHP (8.x) less represented than legacy.	Tends to produce older-style PHP. Needs explicit instructions to use modern patterns (typed properties, enums, fibers).
Swift	Moderate GitHub presence. Apple's documentation is good but closed-ecosystem.	Good for standard iOS/macOS patterns. Can lag behind latest Swift features and SwiftUI changes.
Kotlin	Growing presence. Benefits from Java training data overlap.	Good, especially for Android and Spring Boot. Sometimes produces Java-ish Kotlin unless prompted for idiomatic style.

Language	Why It's Weaker	Practical Notes
Shell / Bash	Heavily represented in training data. But shell scripting is full of edge cases and platform differences.	Good for common patterns. Dangerous for complex scripts — subtle quoting, escaping, and portability issues. Always test.

Tier 3: Functional but unreliable (limited training data or niche)

Language	Why It's Weaker	Practical Notes
Scala	Moderate representation. Complex type system creates many valid approaches.	Can produce working code but may not be idiomatic. Struggles with advanced type-level programming.
Elixir / Erlang	Small community, limited training data. Functional + actor model is a niche paradigm.	Can produce basic code but often misses OTP patterns, supervision trees, and idiomatic approaches.
Haskell	Small training corpus. Advanced type system. Academic-heavy documentation.	Produces basic Haskell but struggles with monads, type classes, and advanced patterns.
R	Moderate data science representation. Fragmented ecosystem (base R vs tidyverse).	Good for standard tidyverse/ggplot patterns. Weaker for base R and statistical modeling edge cases.
Perl	Legacy presence in training data, but declining community.	Can produce Perl but often outdated style. Regex handling is actually quite good due to Perl's regex dominance in training data.
Lua	Limited representation. Mostly game development (Love2D, Roblox) and embedded contexts.	Basic functionality works. Domain-specific Lua (Neovim config, game scripting) varies by how well that domain is represented.

Tier 4: Unreliable (very limited training data)

Language	Why It's Weak	Practical Notes
COBOL	Minimal modern training data. Legacy systems are proprietary and rarely on GitHub.	Can produce syntactically valid COBOL but will miss mainframe conventions, JCL, CICS, and real-world patterns. Hallucination risk is very high.
Fortran	Limited modern representation. Scientific computing niche.	Basic Fortran works. Modern Fortran (2008+) features unreliable.

Language	Why It's Weak	Practical Notes
Ada	Very small training corpus. Defense/aerospace niche.	High hallucination risk. Do not trust without extensive verification.
Assembly	Architecture-specific, minimal in training data relative to size of language space.	Can produce basic x86 assembly. Unreliable for ARM, RISC-V, or optimized routines.
Prolog / Datalog	Very small training corpus. Logic programming paradigm is rare.	Can produce basic Prolog but struggles with complex unification and cut semantics.
APL / J / K	Extremely niche. Almost no training data.	Essentially hallucinating. Do not use.

By Engineering Domain

Front-End vs Back-End

Domain	Capability Level	Why
Front-end (React, Vue, Angular)	⭐⭐⭐⭐⭐ Excellent	Massive training data: millions of tutorials, component libraries, Stack Overflow answers. Highly template-based patterns (component → props → render → state). React especially dominant.
CSS / Styling	⭐⭐⭐⭐ Very Good	Strong pattern matching for common layouts. Struggles with subtle cross-browser issues and complex responsive designs. Tailwind is better than raw CSS because utility classes are more predictable tokens.
Back-end APIs (REST, GraphQL)	⭐⭐⭐⭐⭐ Excellent	Extremely well-represented. CRUD patterns, middleware, routing, auth — all have overwhelming statistical signal.
Database / ORM	⭐⭐⭐⭐ Very Good	Standard ORM patterns (Prisma, SQLAlchemy, ActiveRecord) are very strong. Raw SQL is good for common queries. Complex migrations and performance tuning are weaker.

Domain	Capability Level	Why
Infrastructure / DevOps	⭐⭐⭐⭐ Very Good	Terraform, Docker, Kubernetes, CI/CD configs are well-represented. But infra is environment-specific — the model can't know your AWS account structure.
Systems programming	⭐⭐⭐ Good	Can write system-level code but struggles with concurrency edge cases, memory safety in C/C++, and platform-specific behavior.
Embedded / IoT	⭐⭐ Limited	Very hardware-specific. Limited training data for specific MCUs, HALs, and RTOS patterns.
Real-time / Game engines	⭐⭐⭐ Moderate	Unity (C#) and Unreal (C++) have decent representation. Custom engines and performance-critical rendering code are weaker.

Why Front-End Is Disproportionately Strong

Front-end development is arguably the **single best use case** for LLMs because:

- The code is highly visual and template-based (component trees, JSX, CSS classes)
- There are millions of tutorials, blog posts, and examples online
- The patterns are incredibly consistent (state → render → event → update state)
- Libraries like React, Tailwind, and Material UI have extensive documentation
- Results are immediately visually verifiable
- TypeScript provides type constraints that guide generation

Back-end is also strong but has more variance because business logic is domain-specific and less template-like.

By Task Type

Tasks LLMs Excel At (strong patterns, high training data density)

Task	Why It's Strong	Reliability
Boilerplate generation	Highly repetitive, template-based. Exactly what statistical prediction excels at.	⭐⭐⭐⭐⭐ Near-perfect

Task	Why It's Strong	Reliability
CRUD operations	Millions of examples in training data. Create/Read/Update/Delete is the most common pattern in software.	⭐⭐⭐⭐⭐ Near-perfect
Test generation	Training data includes millions of test files. Model has strong signal for assertion patterns, mocking, edge cases.	⭐⭐⭐⭐ Very good (but review edge case coverage)
Code translation	Pattern mapping between languages. Well-represented in training data.	⭐⭐⭐⭐ Very good for Tier 1 languages
Refactoring	Common refactoring patterns (extract function, rename, convert to async) are well-documented.	⭐⭐⭐⭐ Very good
Documentation / comments	The model is literally a language model — explaining things is its core competency.	⭐⭐⭐⭐⭐ Excellent
Regex generation	Huge training data for regex patterns. Model is better at regex than most humans.	⭐⭐⭐⭐⭐ Excellent
Config files	YAML, JSON, TOML configs are highly structured and well-represented.	⭐⭐⭐⭐ Very good
Standard auth flows	OAuth, JWT, session management — extremely common patterns.	⭐⭐⭐⭐ Very good
Type definitions / schemas	Highly structured, deterministic output. Types constrain the model beautifully.	⭐⭐⭐⭐⭐ Excellent

Tasks LLMs Are Moderate At (mixed patterns, context-dependent)

Task	Why It's Mixed	Reliability
Bug fixing	Strong for common bug patterns. Weaker for subtle logic errors that require deep understanding of business context.	⭐⭐⭐ Good (varies by bug type)
Performance optimization	Knows common optimizations (indexing, caching, algorithm complexity). Struggles with profiling-driven micro-optimization and system-specific tuning.	⭐⭐⭐ Moderate
API integration	Strong for well-documented, popular APIs (Stripe, AWS SDK, Google APIs). Unreliable for niche or rapidly-changing APIs — training data may be stale.	⭐⭐⭐⭐ Good (but verify API versions)
Error handling	Knows patterns (try/catch, Result types, error boundaries). But often produces overly generic or	⭐⭐⭐ Moderate (needs specific

Task	Why It's Mixed	Reliability
	incomplete error handling unless explicitly prompted.	prompting)
Concurrency	Knows async/await, promises, goroutines at a pattern level. Struggles with race conditions, deadlocks, and concurrent state management.	⭐⭐⭐ Moderate (dangerous to trust)
Database design	Good for standard normalized schemas. Weaker for complex denormalization decisions, partitioning strategies, and performance-driven schema choices.	⭐⭐⭐ Moderate
Code review	Can spot common issues (missing null checks, unused imports, obvious bugs). Misses subtle architectural concerns and business logic flaws.	⭐⭐⭐ Moderate

Tasks LLMs Struggle With (novel problems, weak patterns)

Task	Why It's Weak	Reliability
Novel algorithm design	No statistical prior for truly new algorithms. The model approximates using nearest known patterns, which may be wrong.	⭐⭐ Low (must verify rigorously)
Complex distributed systems	Consensus algorithms, distributed transactions, partition tolerance — these require deep reasoning about failure modes with limited training examples.	⭐⭐ Low
Security-critical code	The model has seen both secure and insecure code. It can't reliably distinguish which is which. May reproduce common vulnerabilities from training data.	⭐⭐ Low (always security review)
Exact computation	Character counting, precise arithmetic, complex state tracking across long sequences — token prediction is fundamentally not arithmetic.	⭐ Very low (delegate to code execution)
Legacy system migration	Proprietary legacy systems (mainframes, COBOL, AS/400) have minimal training data. Business rules embedded in legacy code are unique to each organization.	⭐ Very low
Hardware-specific code	Device drivers, FPGA, embedded HALs, platform-specific assembly — too specific, too little training data.	⭐ Very low
Regulatory compliance code	HIPAA, PCI-DSS, SOX — the model knows the concepts but can't verify compliance. Requirements are domain-specific and high-stakes.	⭐ Very low (never trust alone)

Task	Why It's Weak	Reliability
Complex state machines	Multi-state systems with intricate transitions and edge cases. Model loses track of state invariants over long sequences.	⭐⭐ Low

---

By Framework / Library Popularity

The model's capability is directly proportional to how much that framework appears in training data:

```
EXCEPTIONAL          VERY GOOD            GOOD                 UNRELIABLE
(millions of          (hundreds of         (tens of             (sparse
 examples)             thousands)           thousands)           training
data)

React                Next.js              SvelteKit            Qwik
Express              NestJS               Remix                Fresh (Deno)
Django               FastAPI              Actix (Rust)         Axum (Rust)
Flask                Spring Boot          Phoenix (Elixir)     Ktor
Rails                ASP.NET Core         Fiber (Go)           Vapor
(Swift)
Vue                  Angular              Gin (Go)             Lucky
(Crystal)
Node.js              Laravel              Chi (Go)             Rocket
(Rust)
jQuery               Prisma               Drizzle
TensorFlow           PyTorch              JAX
AWS SDK              Terraform            Pulumi               CDK (newer)
Jest/Mocha           Pytest               Vitest               Bun test
runner
Tailwind             Bootstrap            Bulma                Open Props
Docker               Kubernetes           Nomad
PostgreSQL           MongoDB              CockroachDB
Redis                Elasticsearch        Meilisearch
GraphQL              gRPC                 tRPC
```

Key pattern: The more "mainstream" the framework, the better the model performs. Newer frameworks with less historical training data are less reliable, even if they're technically superior.

Practical Recommendations

Maximize strengths

- Use LLMs heavily for boilerplate, CRUD, tests, docs, refactoring, translations, and config
- Lean into TypeScript over JavaScript — types improve generation quality significantly
- Use popular, well-documented frameworks where the model has the strongest priors
- For front-end work, LLMs are a massive force multiplier — lean into this

Compensate for weaknesses

- For novel algorithms: write the logic yourself, have the LLM write tests and boilerplate around it
- For concurrency: write the concurrent logic, have the LLM write the sequential parts
- For security: always run SAST/DAST tools. Never trust LLM output for auth, encryption, or access control without expert review
- For niche languages: provide extensive examples and documentation in context. The less training data the model has, the more context *you* need to provide
- For rapidly-changing APIs: always have the model read current documentation (WebFetch) rather than relying on training data

Context-provide for weak areas

- Working in a Tier 3-4 language? Put extensive code examples and API docs in context
- Working with a niche framework? Provide the docs, or tell Claude to fetch them
- Building something novel? Provide architectural diagrams, specs, and reference implementations
- Integrating with a new API? Paste the API docs directly or provide the URL

The model's capability is not fixed — it's a function of what you put in the context window. For weak areas, you can significantly boost performance by providing the information the training data lacked.